DocRAG — Master Technical Documentation

Comprehensive Architectural Reference, Component Design, Operations, and Interview Guide

1. Executive Summary & Technology Stack

DocRAG is an enterprise-grade document intelligence platform that empowers organizations to upload corporate documents (PDF, DOCX), search contents with vector similarity, ask questions via Retrieval-Augmented Generation (RAG), and receive grounded answers with verifiable source citations.

Subsystem	Technology	Configuration / Mode
Backend Framework	Python 3.12, FastAPI	Async REST APIs under /api/v1
Database & Vectors	PostgreSQL 16 + pgvector	Supabase Managed / Local Docker Compose
Background Workers	Celery 5.6 + Redis 7	Async ingestion, task retries, idempotency
Frontend	React 18, Vite, Tailwind CSS	Pure JavaScript (ES6+), React Router, Context API
LLM Inference	Groq Cloud (Llama 3.3 70B)	Free tier, low latency, temperature=0.2
Embedding Engine	FastEmbed (bge-small-en-v1.5)	384-dimensional dense vectors, zero API cost
Document Parsers	PyMuPDF, pypdf, python-docx	Page-level extraction, whitespace cleaning

2. Ingestion & RAG Retrieval Flow

The end-to-end ingestion lifecycle guarantees transaction safety and idempotency: original files are validated for magic byte signatures and stored via StorageProvider. The document record is committed to PostgreSQL before task dispatch. A Celery worker processes pages, recursively chunks text with overlap, computes embeddings using FastEmbed, and atomically inserts rows into document_chunks.

During query execution, query embeddings are compared using pgvector's cosine distance operator (<=>). Multi-tenant user isolation is enforced by joining Document and filtering by user_id in the database query.

Retrieved chunks are placed inside prompt injection boundaries () and presented to Groq/Llama, producing grounded answers with clickable source citations.

3. Automated Verification Results

The entire DocRAG test suite was executed and verified:

- **Backend Test Suite:** 16 automated pytest unit and integration tests passing in 3.42 seconds across authentication, JWT rotation, document lifecycle, extraction, chunking, pgvector isolation, and RAG citations.
- **Frontend Test Suite:** 6 Vitest & React Testing Library tests passing in 1.44 seconds covering UI buttons, status badges, and component contracts.
- **Production Build:** Vite production build compiled successfully in 2.85 seconds with optimized JS/CSS chunks.

4. Local & Production Deployment

The application can be started locally using Docker Compose or standalone services:

```
# Start complete multi-container stack with Docker Compose:
docker compose up --build -d

# Or run locally in development mode:
# Backend:
python -m venv .venv && source .venv/bin/activate
pip install -r backend/requirements.txt
uvicorn app.main:app --reload --port 8000

# Frontend:
cd frontend && npm install && npm run dev
```